

IHK – 7 Algorithmische Hauptmuster

(IHK-Stil)

1. Suche / enthält

```
Funktion enthaelt(A: Array<T>, gesucht: T) → boolean
  n = länge(A)
  für i = 0 bis n - 1
    wenn A[i] = gesucht dann
      return true
    end if
  end for
  return false
end Funktion
```

2. Zählen mit Bedingung

```
Funktion zaehleMitBedingung(A: Array<T>) → integer
  n = länge(A)
  anzahl = 0
  für i = 0 bis n - 1
    wenn <Bedingung für A[i]> dann
      anzahl = anzahl + 1
    end if
  end for
  return anzahl
end Funktion
```

```
Funktion erstelleListe(A: Array<T>) → Liste<T>
  liste = neueListe()
  für i = 0 bis länge(A) - 1
    wenn <Bedingung für A[i]> dann
      liste.hinzufuegen(A[i])
    end if
  end for
  return liste
end Funktion
```

3. Minimum / Maximum / Durchschnitt

```
Funktion findeMinimum(W: Array<Integer>) → Integer
    min = W[0]
    für i = 1 bis länge(W) - 1
        wenn W[i] < min dann
            min = W[i]
        end if
    end for
    return min
end Funktion
```

```
Funktion findeMaximum(W: Array<Integer>) → Integer
    max = W[0]
    für i = 1 bis länge(W) - 1
        wenn W[i] > max dann
            max = W[i]
        end if
    end for
    return max
end Funktion
```

```
Funktion berechneDurchschnitt(A: Array<Integer>) → Real
    wenn länge(A) = 0 dann
        return 0.0
    end if
    summe = 0
    für i = 0 bis länge(A) - 1
        summe = summe + A[i]
    end for
    return summe / länge(A)
end Funktion
```

4. Prüfziffer / Umrechnung

```
Funktion pruefziffer(Z: Array<Integer>, G: Array<Integer>) → Integer
    summe = 0
    für i = 0 bis länge(Z) - 1
        summe = summe + Z[i] * G[i mod länge(G)]
    end for
    return summe mod 10
end Funktion
```

```
Prozedur Umrechnung(dezimalzahl: Integer)
    wenn dezimalzahl = 0 dann
        ausgabe "0"
        return
    end if

    ergebnis = ""
    n = dezimalzahl

    solange n > 0
        rest = n mod 2
        ergebnis = konvertiere(rest) + ergebnis
        n = n / 2
    end solange

    ausgabe ergebnis
end Prozedur
```

5. Sortieren (einfacher Algorithmus)

```
Prozedur sortiere(A: Array<Integer>)
  n = länge(A)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn A[j] > A[j + 1] dann
        tausche A[j], A[j + 1]
      end if
    end for
  end for
end Prozedur
```

6. Sortieren mit Vergleichsfunktion

```
Prozedur sortiere(L: Array<T>, cmp: Funktion(T, T): Integer)
  n = länge(L)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn cmp(L[j], L[j + 1]) > 0 dann
        tausche L[j], L[j + 1]
      end if
    end for
  end for
end Prozedur
```

```
Funktion vergleicheMitarbeiter(m1: Mitarbeiter, m2: Mitarbeiter) → Integer
  wenn m1.name < m2.name dann
    return -1
  sonst wenn m1.name > m2.name dann
    return 1
  sonst
    return m1.id - m2.id
  end if
end Funktion
```

7. Statistik (2D-Array)

```
Funktion statistik(Daten: Array<Array<Integer>>, limit: Integer) → Statistik
    min = +unendlich
    max = -unendlich
    liste = neueListe()

    für i = 0 bis länge(Daten) - 1
        hatLimit = false
        für j = 0 bis länge(Daten[i]) - 1
            wert = Daten[i][j]

            wenn wert < min dann
                min = wert
            end if
            wenn wert > max dann
                max = wert
            end if
            wenn wert > limit dann
                hatLimit = true
            end if
        end for
        wenn hatLimit dann
            liste.hinzufuegen(i)
        end if
    end for

    return neueStatistik(min, max, liste)
end Funktion
```